



ISACA 2026  
**NORTH AMERICA  
CONFERENCE**

Las Vegas | Virtual | 6-8 May

# The Database Blind Spot: Privilege, Audit, and Compliance Risk in the Data Layer

Erik Brandsberg, CTO, Heimdall Data



# DISCLAIMER

Presentations are intended for educational purposes only and do not replace independent professional judgment. Statements of fact and opinions expressed are those of the presenters individually and, unless expressly stated to the contrary, are not the opinion or position of ISACA or any other co-sponsors. ISACA does not endorse or approve, and assumes no responsibility for, the content, accuracy or completeness of the information presented.

Attendees should note that sessions may be audio- or video-recorded and may be published in various media, including print, audio and video formats without further notice. The presentation template and any media capture are subject to copyright protection.

©2026 ISACA or its affiliates. The ISACA logo and other trademarks are proprietary. All rights reserved.

# Learning Objectives

1. Describe how relational database access controls differ from traditional IAM entitlement models.
2. Identify structural factors within databases that complicate effective privilege aggregation and auditing.
3. Evaluate how native database behaviors can create audit visibility gaps and impact segregation of duties, traceability, and compliance validation.
4. Assess database environments for governance blind spots, including configuration drift and identity-altering constructs that may not be captured in standard access reviews.



# The Governance Assumption

“We know who CAN access what based on roles and grant tables.”

**Assumes these are sufficient to validate *effective* access**

- User & Role Access Reviews
- Role & Privileged Access Certifications
- Grant Table Queries
- IAM Integrations and Controls

# Implicit IAM Model Behind Access Reviews

## Traditional IAM assumes:

- Access is explicitly granted
- Permissions can be fully enumerated
- Effective access = sum of roles + grants
- Access is stable between reviews
- Identity = actor of record

## In relational databases:

- Access may be implicit (ownership, execution context)
- Privilege is not fully enumerable
- Effective access is resolved at runtime
- Access can change without identity changes (drift)
- Identity may be abstracted (proxy users, apps)

# Where Database Access Breaks Compliance Frameworks

## What Frameworks Expect:

### SOX (Financial Controls)

- Segregation of duties enforced
- Controlled access to financial data
- Evidence of who can modify/report financial records

### SOC 2 (CC6 / CC7 – Logical Access & Change Monitoring)

- Access is authorized and least privilege
- User activity is attributable to individuals
- Changes to access are monitored and reviewed

### ISO 27001 (Annex A.9 – Access Control)

- Access rights are provisioned and reviewed
- Privileges are restricted and controlled
- Access is removed when no longer required

### NIST (AC / IA / AU Controls)

- AC: Enforce authorized access
- IA: Strong identity and authentication
- AU: Traceable, attributable audit logs

## Database Realities:

### Segregation of Duties (SOX)

- Execution context (definer rights) bypasses role restrictions
- Object ownership allows indirect privilege escalation
- No role violation required for control failure

### Logical Access Integrity (SOC 2)

- Proxy users and service accounts obscure identity
- Runtime role activation (SET ROLE) not reflected in reviews
- Effective access differs from approved access

### Access Control Enforcement (ISO 27001)

- Privileges are not fully enumerable (implicit via ownership, inheritance)
- Access paths created via procedures/views not captured in reviews
- Revocation does not remove control if ownership persists

### Auditability & Traceability (NIST)

- Actions attributed to application accounts, not individuals
- Cross-boundary queries (FDW, linked servers) escape audit scope
- Logs reflect execution context—not originating identity

# Structural Complexity in Database Privilege Evaluation

## What Actually Determines Effective Access?

- Object ownership
- Recursive role inheritance
- Session state
- Execution context
- Cross-boundary constructs

These constructs are evaluated by the database engine at *runtime* — and are not fully reflected in standard entitlement reports.

Privilege evaluation behavior varies by database engine and even *version*.

# When Access Assurance Becomes a Gamble

Point-in-time entitlement reporting

Manual certification processes

No runtime privilege resolution

No configuration integrity validation

No continuous drift detection

If effective access is determined at runtime AND

configuration can change post-review THEN

an approved access review is an assumption of stability, not verified assurance.

You are betting that the control model has not changed.

# When The Evidence Gap: “Reported” Vs. “Effective Access”

## Reported Access

- Grants + role membership lists
- Snapshot in time
- Often incomplete for indirect/implicit paths

## Effective Access

- What a user/service can actually do in a real session
- Depends on runtime context (inheritance, ownership, activation, execution context)

- Audit certifications often validate reported access - while risk is driven by effective access.

# The Evidence Gap: “Reported” Access vs “Effective” Access

## Why Runtime Drift Detection is Needed

- Access assurance becomes a gamble when:
- Reviews rely on point-in-time exports (roles/grants)
- Privileges are resolved at runtime by the database engine
- The control model changes after review (drift)
- Key access paths are implicit or indirect (not in the report)

Bottom line:

“Approved access”  $\neq$  “Assured access” without runtime-aware evidence and drift detection.

# Why Point-in-Time Entitlement Reports Fail

## What Auditors Usually Receive

Typical artifacts used during access reviews:

- Grant table exports
- Role membership lists
- “Who has SELECT/UPDATE” reports
- IAM entitlement exports
- Privileged role inventories

## Point-in-time reports typically omit or under-sample:

- Recursive role inheritance
- Nested roles create transitive privilege paths.
- Object ownership
- Owners retain control even when grants appear restricted.
- Role activation and session state
- Privileges may exist but are activated dynamically.
- Execution context changes
- Procedures and views may execute with elevated privileges.
- Cross-boundary constructs
- Linked servers, FDWs, external tables, or synonyms extend access beyond the reviewed system.

# Database access models change as part of normal operations

After an access review is approved, the control model may change through routine activity such as:

- Role membership changes
- New privileges granted to existing roles
- Ownership transfers of database objects
- Deployment of new procedures, views, or triggers
- Creation of cross-boundary constructs (FDW, linked servers, external tables)
- Default role or session configuration changes

None of these require malicious activity — they are normal database lifecycle events.

# What Auditors Actually Need to Prove

- The audit objective is not to validate a report. It is to validate effective access.

A meaningful access control review must answer:

***At time  $T$  - and continuously between  $T$  and  $T+n$  - can identity  $X$  perform action  $Y$  on data  $Z$ , directly or indirectly?***

This requires evaluating:

- Structural privilege relationships (roles, ownership, inheritance)
- Runtime privilege resolution (execution context, session state)
- Cross-boundary access paths (linked systems, external tables, FDWs)
- Changes to the control model over time (configuration drift)

Effective access assurance requires more than static entitlement reports.

# Object Ownership: Privilege Beyond the Grant Tables

Database object ownership introduces implicit control that may not appear in entitlement reports.

In most relational databases, the owner of an object can:

- Modify or redefine the object
- Grant privileges on the object to other identities
- Change execution behavior (views, procedures, triggers)
- Replace or redirect access paths to underlying data

These capabilities may exist without explicit privileges appearing in grant tables.

# Object Ownership, Continued

## Governance Risk

Ownership can undermine segregation-of-duties controls when developers or administrators own sensitive objects.

Example:

- A developer owns a reporting view used by analysts.
- The developer can redefine the view to expose additional data without changing analyst privileges.
- The access review still appears valid.

Ownership controls object behavior, not just access to the object.

# Ownership and Ephemeral Access: Operational Risk

**Many organizations use ephemeral or just-in-time privileged access to limit standing permissions**

Ownership introduces a complication for this:

If temporary access allows a user to create or own database objects, removing that access may leave:

- Orphaned objects
- Broken dependencies (views, procedures, triggers)
- Configuration artifacts owned by an identity that no longer exists
- Operational pressure to restore elevated privileges

## **Governance Implication**

- Revoking privileged access may not safely return the system to its previous state if ownership relationships were created during the session.

Privilege revocation does not always remove control if ownership relationships remain.

# Recursive Role Inheritance: Privilege Expansion Through Structure

Roles can inherit from other roles, creating transitive privilege paths that are not visible in direct membership lists.

In many database systems:

- Roles can be nested within other roles
- Membership can be indirect (Role A → Role B → Role C)
- Privileges accumulate across the chain

## Key Risk

Access reviews that evaluate only direct role membership will under-sample actual privilege.

## Example

User is assigned to Reporting\_Role

But:

Reporting\_Role → Analytics\_Role →  
Data\_Admin\_Role

Data\_Admin\_Role has write access to sensitive tables.

The user inherits this access indirectly — even if it is not visible in the report.

**Some identity platforms (e.g., Okta) avoid nested group membership to reduce evaluation complexity.**

# Session State and Role Activation: Privilege That Isn't Always Visible

Some privileges exist but are only active at runtime based on session behavior.

In many database systems:

- Roles may not be active by default
- Users can activate additional roles during a session (e.g., SET ROLE)
- Default role configurations vary by user and environment
- Session context can influence which privileges are applied

## Key Risk

A user may appear to have limited access in entitlement reports,

but can activate additional privileges during runtime.

**Effective access includes what a user can activate — not just what is active by default.**

## Example

User is assigned Reporting\_User (default role)

Also has access to Data\_Admin\_Role (not active by default)

During a session:

SET ROLE Data\_Admin\_Role;

User now has elevated privileges not visible in the default state.

## Audit Implication

Access reviews that evaluate only default or active roles at rest may miss privileges that can be activated on demand.

# Execution Context: Code That Runs With Elevated Privileges

Database objects may execute with privileges different from the calling user.

In many systems:

- Stored procedures and functions may run with definer/owner privileges
- Views may expose data through owner-controlled access paths
- The database evaluates permissions based on the execution context, not always the caller
- This behavior is often configurable (definer vs invoker semantics)

## Key Risk

A user with limited privileges can access restricted data by executing objects that run with elevated permissions.

## Example

User has no direct access to Sensitive\_Table

But can execute:

Stored Procedure: GetSensitiveData()

The procedure runs as the owner and returns data from Sensitive\_Table.

## Audit Implication

Access reviews that focus on direct privileges and role membership may miss access paths created through executable objects.

# Proxy Users and Identity Substitution: Access Without Direct Identity

Some database systems allow sessions to operate under a different identity than the authenticated user.

Examples include:

- Proxy users (one user connecting “as” another)
- Application/service accounts acting on behalf of users
- Credential delegation or shared access models

Example

User connects as: User\_A

Database allows: User\_A to proxy as Reporting\_User

Session executes as: Reporting\_User (not User\_A)

All database activity appears as:

Reporting\_User

Result: User\_A does not appear in grant tables or database audit logs

## Key Risk

The identity performing actions in the database may not match the originating user.

# Cross-Boundary Access: Privilege That Extends Beyond the System

Databases frequently expose access paths that extend beyond their own security boundary:

Configuration artifacts include:

- Synonyms / views referencing external schemas
- Linked servers / database links
- Foreign Data Wrappers (FDWs)
- External tables / data virtualization layers

What appears “local” may resolve to remote data sources at runtime.

Effective access may include

- Remote databases
- Different security domains
- Systems not included in the audit scope

Example

User has SELECT on a local view Reporting\_View

View resolves to External table via FDW → remote finance database

User can now access sensitive financial data

- Without direct privileges in that system
- Not visible in local grant reports

“Who can access this database?” is the wrong question

Must instead ask: “Where can access from this database reach?”

# Controls Are Evaluated Individually — Access Is Not

Access reviews validate:

- Roles
- Grants
- Ownership
- Permissions

But databases evaluate access at runtime using:

- Ownership
- Execution context
- Role inheritance
- Session behavior

Each control may be:

- Approved
- Audited
- Individually compliant

But:

Effective access = interaction of controls

Compliant components can still produce non-compliant outcomes

# Example: SoD Failure Without Any Single Control Violation

Developer:

- No direct access to sensitive data

Analyst:

- Read-only reporting role

Grant tables:

- Appear correct

Access review:

- Approved

- Developer owns reporting schema
- Creates view (definer rights)
- View accesses sensitive tables
- Analyst queries the view
- View runs with elevated privileges

## Result

- No role violation
- No grant change
- No alert in access review

# What Needs to Be Monitored to Detect Meaningful Change?

You don't need full resolution to start—only enough signal to detect when the control model has changed.

## 1. Identities & Substitution (Who is actually acting?)

- Creation/removal of users and service accounts
- Changes to authentication methods (local vs federated)
- Proxy users / identity substitution mechanisms
- Shared or application accounts
- IDP / IAM integrations (SSO, group sync, token-based access)

## 2. Role & Privilege Changes (Structural Access)

- Role membership changes (direct or nested)
- Grants/revokes on sensitive objects
- Creation of new roles or privilege paths

## 3. Ownership Changes (Control Over Behavior)

- Ownership transfer of tables, views, procedures
- Creation of new owned objects in sensitive schemas

## 3. Executable Objects (Hidden Access Paths)

New or modified:

- Stored procedures
- Functions
- Views
- Security definers

## 4. Cross-Boundary Constructs (Scope Expansion)

Creation/modification of:

- FDWs / database links
- External tables
- Synonyms

## 5. Configuration Drift Over Time (Integrity)

Changes to:

- Role defaults / activation behavior
- Security-relevant configuration parameters

Complete evidence starts with documentation of all these points

# What an Effective Evidence Pipeline Looks Like

Organizations should be able to produce continuous, verifiable evidence of effective access-not just point-in-time reports.

## 1. Baseline Capture (Initial State)

Snapshot of:

- Identities and authentication mappings (including IDP integration)
- Roles, grants, and privilege structures
- Ownership of sensitive objects
- Executable objects and cross-boundary constructs

## 2. Continuous Change Detection

Track changes to:

- Identities and role membership
- Ownership and object definitions
- Executable objects (procedures, views, triggers)
- Cross-boundary access paths
- Security-relevant configuration

## 3. Context-Aware Evaluation

- How privileges resolve at runtime
- Whether changes introduce new access paths impact on:
  - Segregation of duties
  - Data exposure
  - Privilege boundaries

## 4. Evidence Retention & Traceability

- Historical record of changes
- Mapping of changes to affected access paths
- Ability to answer:
  - “What changed, when, and what did it enable?”

## 5. Review & Attestation

- Meaningful structural changes occur
- Provide:
  - Evidence that changes were evaluated
  - Confirmation that controls remain valid

# Common Approaches to Access Evidence - and Their Limitations

## Option 1 - Manual / Ad Hoc Collection

- DBA queries, screenshots, exports
- One-time analysis for audit

### Strengths

- Low tooling requirements
- Flexible, targeted

### Limitations

- High effort
- Not repeatable
- No visibility between reviews
- Does not detect drift

## Option 2 – Periodic Extraction / Offline Analysis

- Scheduled exports of roles, grants, and configurations
- Compared periodically

### Strengths

- More structured
- Some trend visibility

### Limitations

- Still point-in-time
- Changes between intervals may be missed
- Limited understanding of runtime behavior
- Lag between change and detection

# Common Approaches to Access Evidence, continued

## Option 3 - Continuous Monitoring / Runtime-Aware Controls

- Ongoing tracking of changes
- Context-aware evaluation of access paths

### Strengths

- Detects drift as it occurs
- Aligns with how access is actually resolved
- Enables continuous assurance

### Limitations

- Requires integration across systems
- More complex to implement
- Organizational maturity required

Most organizations operate in Option 1 or 2, but attempt to claim the assurance level of Option 3

# A Practical Workflow for Evaluating Effective Access

## Step 1 – Define Scope

- Identify Sensitive data sets (PII, financial, regulated)
- High-risk identities (privileged users, service accounts)

## Step 2 – Establish Identity Context

- Understand User vs service identities
- Proxy usage / shared accounts
- IDP / IAM integration model

## Step 3 – Resolve Structural Access

- Evaluate Role inheritance (including nested roles)
- Direct and indirect privilege paths
- Ownership of sensitive objects

## Step 4 – Evaluate Execution Paths:

- Identify stored procedures, functions, views
- Execution context (definer vs invoker)
- Determine what can be accessed indirectly

## Step 5 – Assess Cross-Boundary Reach:

- **Identify external data sources (FDW, linked servers, etc.)**
- Determine whether access extends beyond audit scope

## Step 6 – Evaluate Change Over Time:

- Review: Role, ownership, and object changes
- Determine: Whether access has changed since last review

If any of these steps cannot be performed, effective access cannot be validated.

# How to Apply This Without Boiling the Ocean

Start with risk—not completeness.

## 1. Focus on High-Value Data First

Prioritize:

- Financial data
- PII / regulated data
- Sensitive operational systems

## 2. Start With High-Risk Identities

- Privileged roles
- Service / application accounts
- Accounts with write or administrative capability
- Risk is concentrated—not evenly distributed

Not all data requires the same level of assurance

### 3. Target Critical Access Questions

Focus on:

- Who can modify sensitive data
- Who can exfiltrate data
- Who can alter control structures

Don't try to model every access path initially

### 4. Expand Iteratively

Start with:

- One system
- One dataset
- One identity group

Then expand scope over time

Build coverage progressively

## 5. Align With Existing Audit Cycles

Integrate with:

- Access reviews
- SOX / SOC2 controls

Use drift signals to trigger deeper reviews

Enhance existing controls—don't replace them

# Closing the Database Blind Spot

How databases actually determine access:

- Ownership
- Role inheritance
- Session state
- Execution context
- Cross-boundary constructs

How access is typically audited:

- Point-in-time reports
- Static entitlement models
- Manual certification

The gap:

What is reviewed is often not what is enforced

What effective assurance requires:

- Complete structural resolution of access
- Visibility into runtime behavior
- Detection of changes over time (drift)

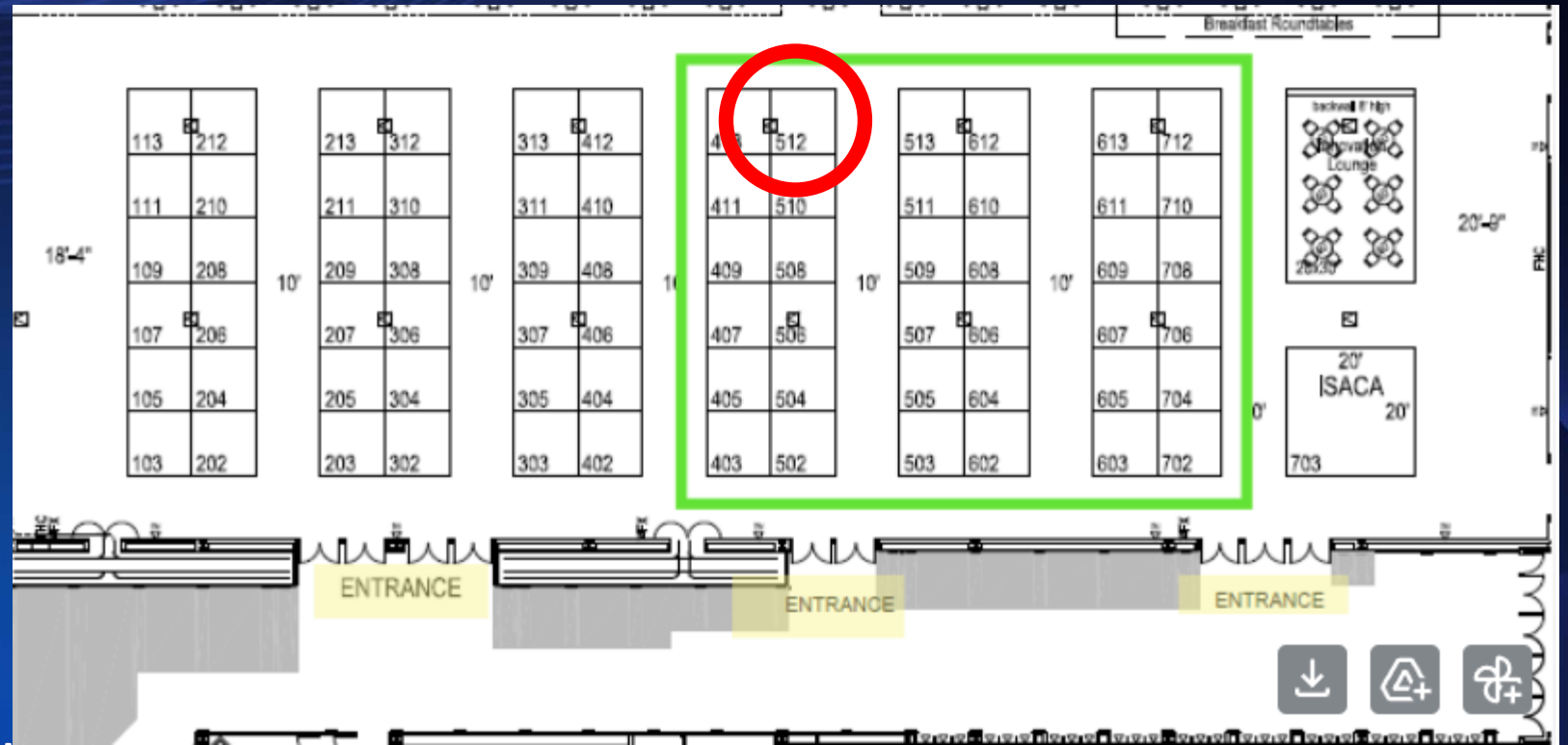
What this means for audits:

- Validate effective access—not just reported access
- Require evidence that reflects how access is actually resolved
- Reassess when the control model changes

If your evidence does not account for how access is actually resolved, you don't have assurance—you have an assumption.

# Please Visit our Booth for More Info on Automating DB Audits

## Booth 512



## Let us show how your customers can provide database access evidence automatically

# Thank you!

We value your feedback!  
Please complete the session  
survey and share your  
thoughts with us.



ISACA 2026  
**NORTH AMERICA  
CONFERENCE**  
Las Vegas | Virtual | 6-8 May